



The fastest, ultra-realistic voice AI platform

# CARTESIA<sup>✦</sup>

01

# Intro



# Zubin Pratap

♦ Head of DevRel, Cartesia

Previously

DevRel Eng Manager @ Chainlink Labs

Software Engineer @ Google

Ex-Founder

Corporate Lawyer for 15 years

<https://linkedin.com/in/zubinpratap>

<https://x.com/@zubinpratap>



# Cartesia is an AI Research & Product Lab

## Model / infrastructure layer

| Model / infrastructure layer                                                     |                                                                                      |                                                        |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------|
| Sonic<br>Text-to-speech                                                          | Ink<br>Speech-to-text                                                                | René<br>Speech-to-speech full duplex model coming 2026 |
| <b>Fastest streaming TTS</b><br>40ms time to first audio                         | <b>Fastest streaming STT</b><br>90ms time to complete transcript                     | → Context-aware responses with long-living memory      |
| <b>Adherence to complex transcripts</b><br>Acronyms, initialisms, IDs            | <b>Optimized for real-world settings</b><br>Background noises, disfluencies, accents | → Context-aware responses with long-living memory      |
| <b>Preferred by human raters</b><br>Naturalness, disfluencies, accent similarity |                                                                                      | → Natural backchanneling and interruptions             |
|                                                                                  |                                                                                      | → Best for use cases needing high emotional awareness  |

## Application layer

### Line

Voice agent platform

### Built for today's AI teams:

Code-first environment to build, deploy, and test your voice agents quickly with one-shot code generations and reasoning templates

### Fully Extensible & Self-Improving:

Custom reasoning & built-in eval framework to help you rapidly iterate

# 02 SSMs





State Space  
Models

The work around SSMs, which allow you to learn extremely long patterns and sequences without growing quadratically in computation, probably is **the next Transformer**.

$O(N^2)$

Jensen Huang  
CEO  
NVIDIA

# State space models (SSMs) are a breakthrough for AGI

> **Transformers** are compute hungry and can't handle large context

> Like humans, **SSMs are low latency** & efficiently handle large context

> **Compute Efficient:** Transformer quality with 10-100x throughput

> **Low Latency:** Enable real-time interaction at human speeds

**Long Context:** Enable lifelong learning and long context models> **Long Context:** Enable lifelong learning and long context models

## Linear-Time Sequence Modeling with Selective State Spaces

Albert Gu\*  
Machine Learning Department  
Carnegie Mellon University  
agu@cs.cmu.edu

Tri Dao\*  
Department of Computer Science  
Princeton University  
tri@tridao.me

### Abstract

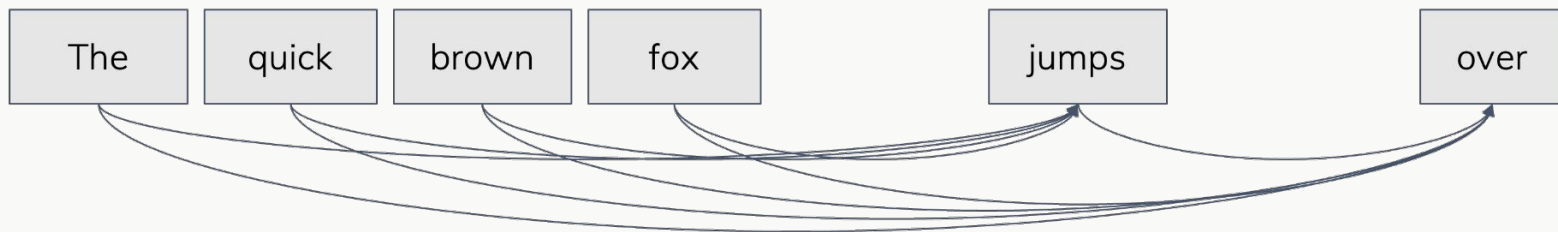
Foundation models, now powering most of the exciting applications in deep learning, are almost universally based on the Transformer architecture and its core attention module. Many subquadratic-time architectures such as linear attention, gated convolution and recurrent models, and structured state space models (SSMs) have been developed to address Transformers' computational inefficiency on long sequences, but they have not performed as well as attention on important modalities such as language. We identify that a key weakness of such models is their inability to perform content-based reasoning, and make several improvements. First, simply letting the SSM parameters be functions of the input addresses their weakness with discrete modalities, allowing the model to *selectively* propagate or forget information along the sequence length dimension depending on the current token. Second, even though this change prevents the use of efficient convolutions, we design a hardware-aware parallel algorithm in recurrent mode. We integrate these selective SSMs into a simplified end-to-end neural network architecture without attention or even MLP blocks (**Mamba**). Mamba enjoys fast inference ( $5\times$  higher throughput than Transformers) and linear scaling in sequence length, and its performance improves on real data up to million-length sequences. As a general sequence model backbone, Mamba achieves state-of-the-art performance across several modalities such as language, audio, and genomics. On language modeling, our Mamba-3B model outperforms Transformers of the same size and matches Transformers twice its size, both in pretraining and downstream evaluation.

### 1 Introduction

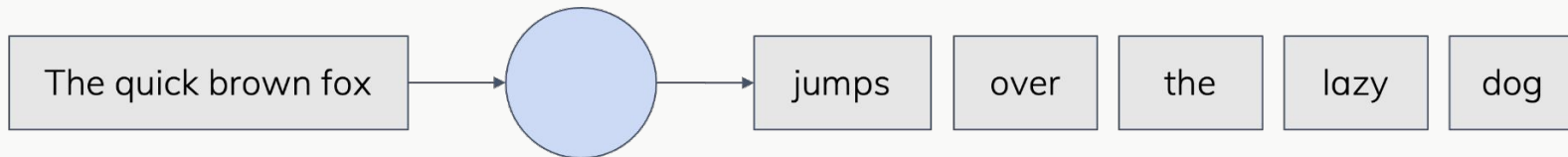
Foundation models (FMs), or large models pretrained on massive data then adapted for downstream tasks, have emerged as an effective paradigm in modern machine learning. The backbone of these FMs are often *sequence models*, operating on arbitrary sequences of inputs from a wide variety of domains such as language, images, speech, audio, time series, and genomics (Sutskever et al., 2014; Dosovitskiy et al., 2020; Oord et al., 2016; Brown et al., 2020; Ismail Fawaz et al., 2019; Poli et al., 2023). While this concept is agnostic to a particular choice of model architecture, modern FMs are predominantly based on a single type of sequence model: the Transformer (Vaswani et al., 2017) and its core attention layer. The efficacy of self-attention is attributed to its ability to route information densely within a context window, allowing it to model complex data. However, this property brings fundamental drawbacks: an inability to model anything outside of a finite window, and quadratic scaling with respect to the window length. An enormous body of research has appeared on more efficient variants of attention to overcome these drawbacks (Tay et al., 2022), but often at the expense of the very properties that makes it effective. As of yet, none of these variants have been shown to be empirically effective at scale across domains.

# State Space Models (SSMs)

Transformers generate **quadratically** by attending to every past token



In SSMs, tokens stream in, update the model's state and are discarded



State is highly compressed, enabling generation in **linear time**



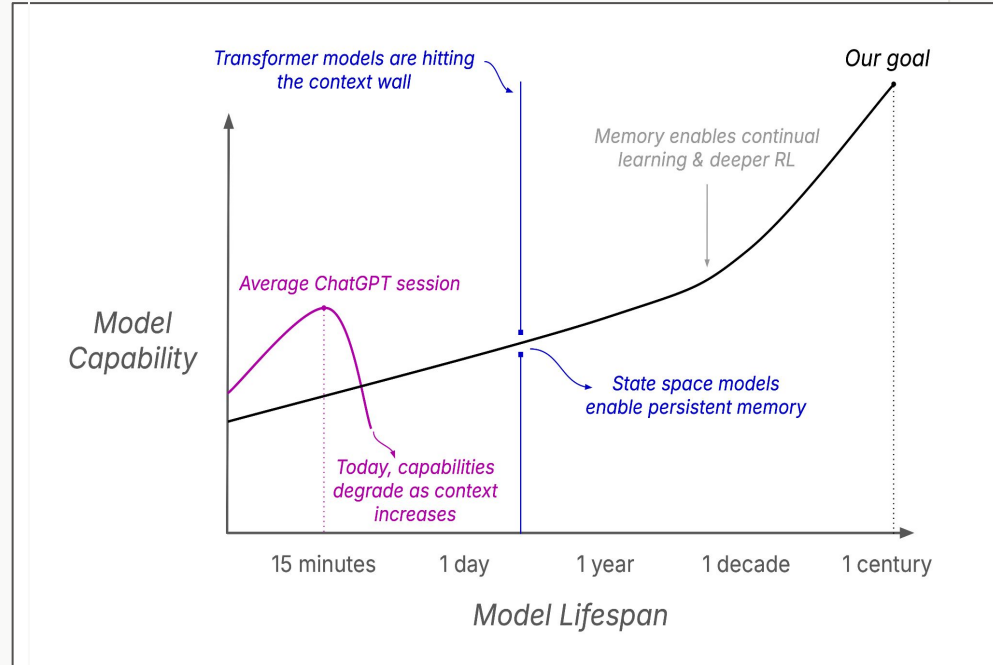
## Voice Is Just the Beginning...

Transformer models hit a context wall. SSMs scale to theoretical infinite context, unlocking persistent memory that grows smarter over time.

Models that don't just respond, but **remember, learn, and adapt**.

Long memory enables:

- **Continual learning** and deeper reinforcement learning
- **Intelligent data handling** in the field
- **Autonomous systems** in high-stakes, rapidly evolving environments
- **Real-time situational awareness** and long-horizon decision-making



# 03

## AI Voice Agents

(that customers trust)

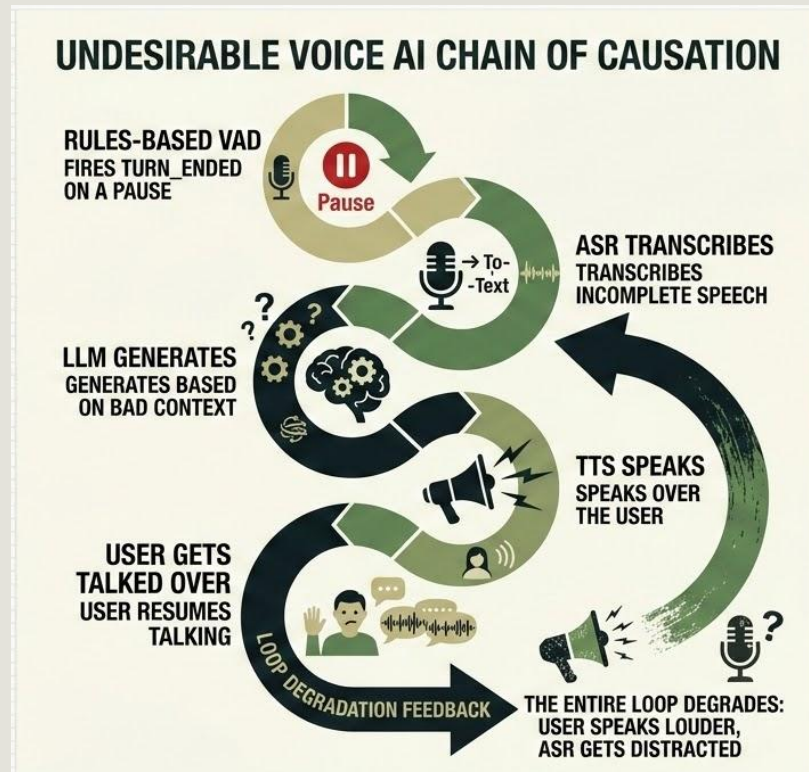
# Voice Agents are not Chatbots with Voice

- > Simple STT → LLM → TTS is great on your laptop, but...
- > On 8khz PSTN (public telephony)?
- > e2e Latency (P95)
- > Conversation is messy (backchanneling, pauses, interruptions, background noise)



# Voice Agents are not Chatbots with Voice Cont

- > Rules based VAD != Interruption Handling
- > Detecting speech is not enough (precision vs recall)
- > Low WER (accuracy) & low latency is not enough either
- > Clear, natural speech is not enough either!
- > Trade offs are real - use case dependent.



# Full Stack Latency Budgets - Part 1

## How Latency is experienced

**< 500ms**

### **Feels Conversational**

Natural interaction with no conscious gap

**500-800ms**

### **Acceptable Delay**

Slight pause, but flows reasonably

**800ms-1.5s**

### **Noticeably Slow**

Lag begins to disrupt natural turn-taking

**> 1.5s**

### **Broken Experience**

Users frequently interrupt or abandon call

# Full Stack Latency Budgets - Part 2

| COMPONENT                         | IDEAL TARGET                                   | NOTES                                                    |
|-----------------------------------|------------------------------------------------|----------------------------------------------------------|
| <b>VAD</b><br>Turn detection      | <b>10–50ms</b>                                 | Endpointing accuracy is the real lever, not raw speed    |
| <b>STT</b><br>Streaming engine    | <b>60–150ms</b>                                | Batch STT adds 600-1200ms — kills the budget outright    |
| <b>LLM</b><br>Time-to-first-token | <b>100–250ms</b>                               | Biggest, least controllable chunk; stack dependent       |
| <b>TTS</b><br>Time-to-first-audio | <b>40–100ms</b><br>Cartesia: 40-60ms           | Cartesia Sonic is fastest production TTS at 40-60ms      |
| <b>Network</b><br>Transport layer | <b>20-60ms</b> WebRTC<br><b>150-700ms</b> PSTN | PSTN telephony transport alone can blow the whole budget |



*Example Use Case*

# Multilingual Product-Support Agent with Escalation



# Building Multilingual AI Voice Agents That Customers Trust

Voice agents that customers trust have to:

- > sound right,
- > understand right,
- > respond right, and
- > recover gracefully

in every supported language.



## **Sonic TTS:**

44 languages, 61 locales, sub-90ms TTFB



## **Ink-2 ASR/STT:**

Super fast, highly accurate, and includes turn detection with semantic endpointing, TTFS 100ms



## **Line Voice Agents**

voice agent infra with LLM reasoning, tool calling, and telephony



# Product-Readiness

The voice agent must:

- > Listen & Understand accurately
- > Respond in the caller's locale (language, accent, normalized)
- > Handle Interruptions + Update state gracefully
- > Call tools reliably & handle failure
- > Be compliant, handle PII etc
- > Escalate intelligently
- > O11y



## The Models Matter

Accuracy, speed, availability, precision vs recall



## ASR

Detect speech, detect silence, understand accents, transcribe fast, handle interruptions



## TTS

localization/multilingual, accents, normalization, prosody and tone

# Product-Readiness (contd)

## Locale Awareness:

Multilingual TTS. Language + localization. Sounds native.

X-lingual: Same voice sound like itself in other languages.

## Reliability:

Normalization, alphanumerics, degraded audio, background noise, consistency across sample rates, encoding types, spliced audio, clipped syllables,



## The Models Matter

Accuracy, speed, availability, precision vs recall



## ASR

Detect speech, detect silence, understand accents, transcribe fast, handle interruptions



## TTS

localization/multilingual, accents, normalization, prosody and tone

# 04 Sonic (TTS)



05

Ink-2 (ASR/STT)

# 06

# On Vapi

# 07 Getting It Right



## From Prototype to Production: A Checklist

- > Define Worst-Case Call Scenario
- > Map Use Cases to Core Metrics
- > Set Component Latency Budgets (ASR, LLM, TTS)
- > Test with Real Phone Call Audio
- > Select Deployment: Managed vs Self-Hosted
- > Localize & Verify Voice Identity
- > Define Human Escalation Paths
- > Validate Structured Data & Interruptions
- > Establish Production Monitoring Framework



**Try for free + API Keys:**

[play.cartesia.ai](https://play.cartesia.ai)

**Docs:**

[docs.cartesia.ai](https://docs.cartesia.ai)

**Client Libraries:**

<https://github.com/cartesia-ai/cartesia-js>

<https://github.com/cartesia-ai/cartesia-python>

**Credits Coupon - 1 Month of Pro**  
**CARTESIA\_VAPI2026\_JL5PP**

**Docs Skill**

[docs.cartesia.ai/skill.md](https://docs.cartesia.ai/skill.md)

**Blog**

<https://cartesia.ai/blog>

# CARTESIA<sup>✦</sup>